Assignment-2

Verilog Lab Questions

Name: Darisy Sreetej
Roll Number: EE25BTECH11018
Course: Digital Systems Lab
Code: EE1501
Institution: IIT HYDERABAD

Date of Submission: April 4, 2026

Academic Year 2025–26

1 Question 1

16-Bit Vector Reversal

```
module bit_reverse(
    input [15:0] in,
    output [15:0] out
    );
    assign out = { in[0], in[1] , in[2] , in[3] , in[4] , in[5] , in[6] , in[7] , in[8] , in[9] , in[10] , in[11] , in[12] , in[13] , in[14] , in[15]};
endmodule
```

Figure 1: design code

```
module tb_bit_reverse;

    reg  [15:0] in;
    wire [15:0] out;

    bit_reverse uut ( .in(in) , .out(out) );

    initial begin
        $display(" in                 out");
        $monitor("%b  %b", in, out);

        in = 16'b1010101010101010; #10;
        in = 16'b1111000011110000; #10;
        in = 16'b1100110000110011; #10;
        in = 16'b0000000000000001; #10;

        $finish;
    end
endmodule
```

Figure 2: testbench-code

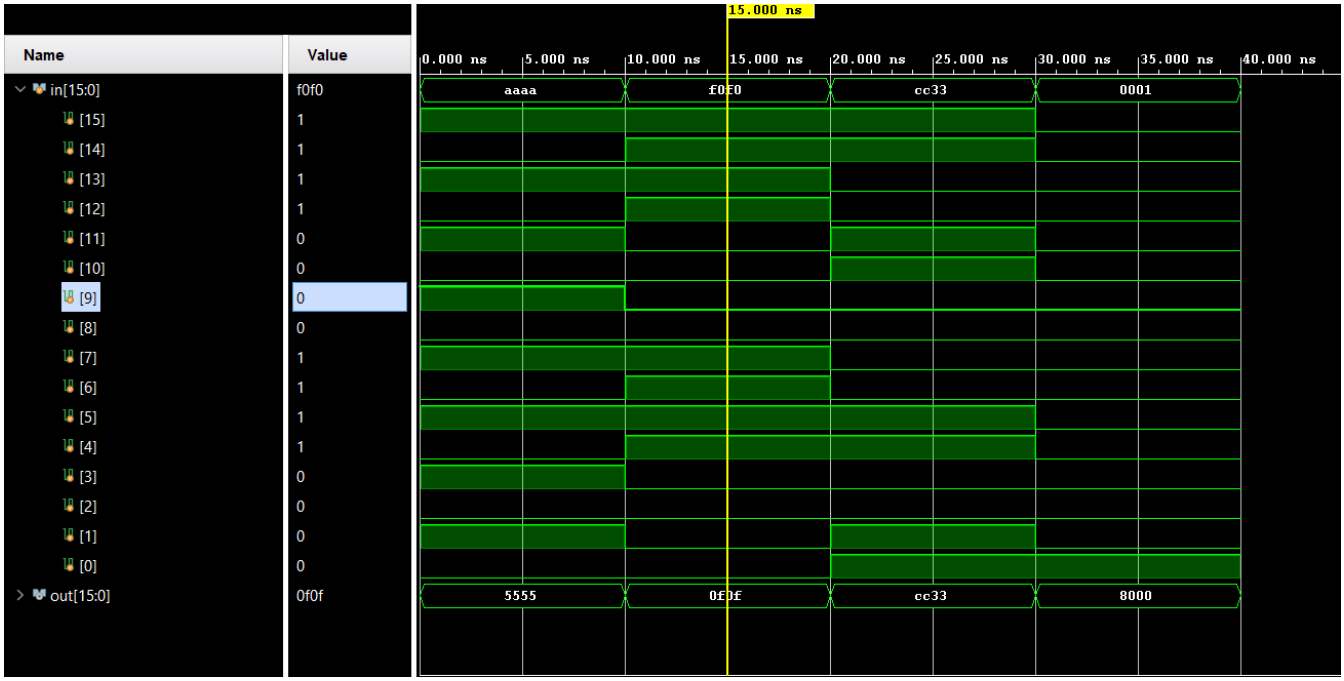

Figure 3: Waveform-1

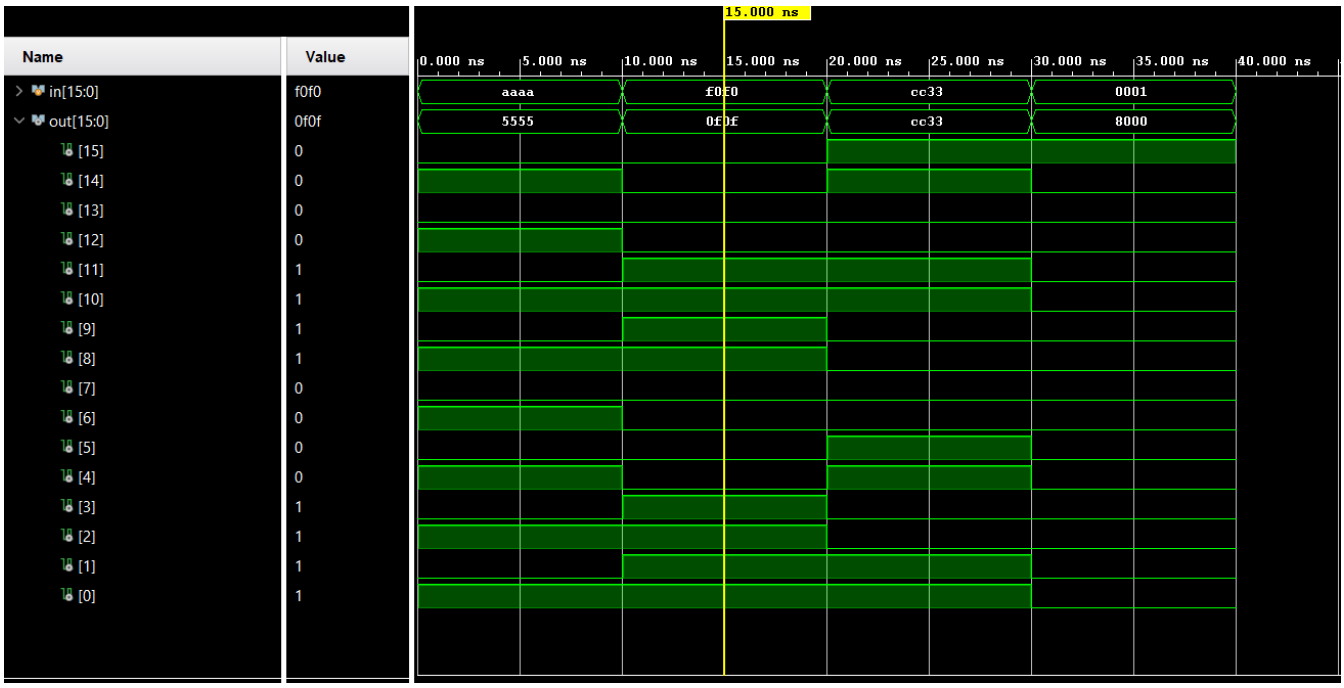

Figure 4: Waveform-1(1)

Explanation

- The bit reversal module takes a 16-bit input **in** and produces a 16-bit output **out** by reversing the order of bits.
- This is implemented using the concatenation operator, where the least significant bit of **in** becomes the most significant bit of **out**, and so on.

- In the testbench , after each assignment, a delay of **#10** is used to observe the corresponding output clearly.
- The **$monitor** statement displays the input and output values during simulation, and the simulation ends using **$finish**.

Observations

The waveform shows that the output **out** is the bit-reversed version of the input **in**.

- For **in** = **1010101010101010**, the output **out** = **0101010101010101**.
- For **in** = **1111000011110000**, the output **out** = **0000111100001111**.
- For **in** = **1100110000110011**, the output **out** = **1100110000110011** (same pattern).
- For **in** = **0000000000000001**, the output **out** = **1000000000000000**.

Thus, the waveform verifies that the bit reversal operation is performed correctly.

2 Question 2

Wide 2-to-1 Multiplexer

```
module mux_2to1(
    input [31:0] a,
    input [31:0] b,
    input  sel,
    output [31:0] out
    );

    assign out = (sel) ? b : a;

endmodule
```

Figure 5: design code

```
module tb_mux_2to1_32bit;

reg  [31:0] a, b;
reg  sel;
wire [31:0] out;

mux_2to1 uut (.a(a) , .b(b) , .sel(sel) , .out(out) );

initial begin

    $display(" sel      a       b        out ");
    $monitor(" %b    %h    %h    %h", sel, a, b, out);

    a = 32'hAAAAAAAA;
    b = 32'h55555555;
    sel = 1'b0;    #10;
    sel = 1'b1;    #10;

    a = 32'hFFFF0000;
    b = 32'h0000FFFF;
    sel = 1'b0;    #10;
    sel = 1'b1;    #10;

    $finish;

end

endmodule
```

Figure 6: testbench code

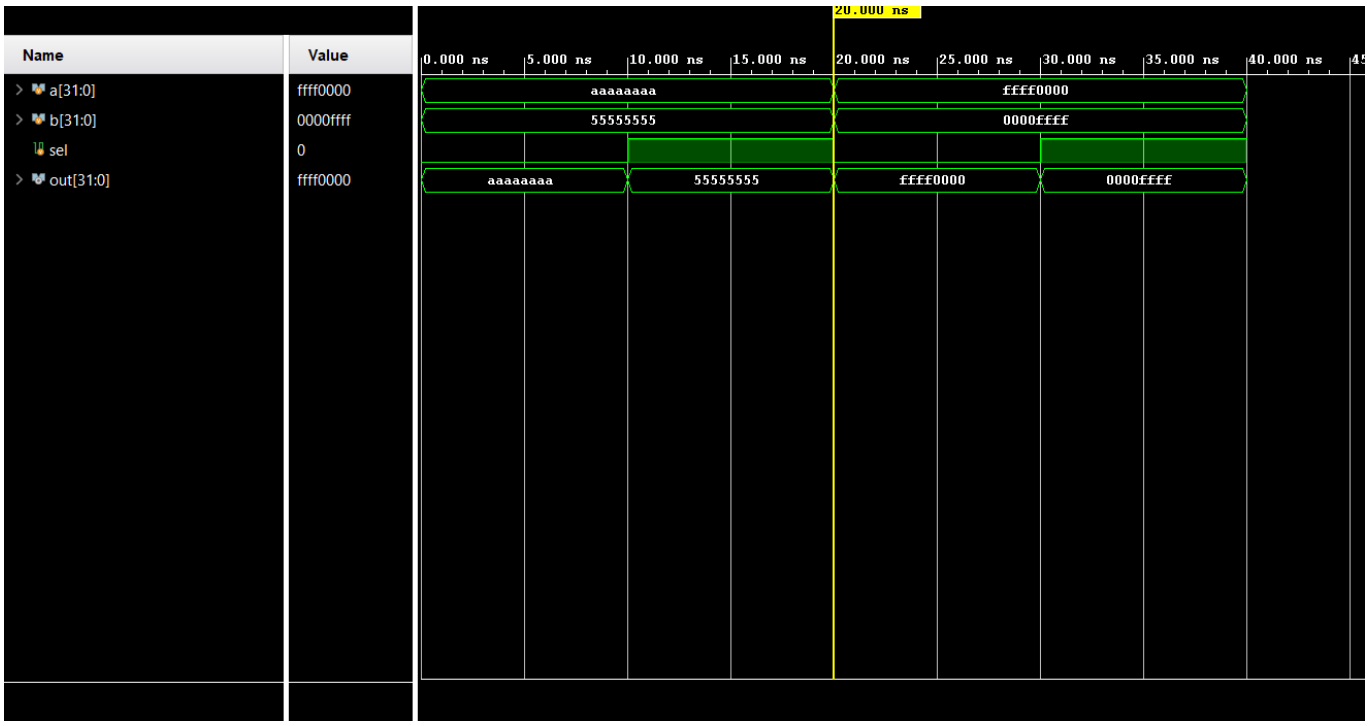

Figure 7: Waveform -2

Explanation

- The 32-bit 2-to-1 multiplexer is implemented using a continuous assignment statement with the conditional operator. Based on the value of the select signal **sel**, the output **out** is assigned either input **a** or input **b**.
- When **sel** = **0**, the output follows **a**, and when **sel** = **1**, the output follows **b**.
- A delay of **#10** is introduced between changes in the testbench to clearly observe the switching behavior in the waveform.
- The **$monitor** statement displays the input and output values during simulation, and the simulation ends using **$finish**.

Observations

The waveform shows the selection of inputs based on the value of **sel**.

- At 0–10 ns: **sel** = **0**, hence **out** = **a** = **AAAAAAAA**.
- At 10–20 ns: **sel** = **1**, hence **out** = **b** = **55555555**.
- At 20–30 ns: **sel** = **0**, hence **out** = **a** = **FFFF0000**.
- At 30–40 ns: **sel** = **1**, hence **out** = **b** = **0000FFFF**.

Thus, the waveform verifies that the multiplexer correctly selects between the two 32-bit inputs based on the select signal.

3 Question 3

Minimum of Four Numbers

```
module min_value(
    input [7:0] a,
    input [7:0] b,
    input [7:0] c,
    input [7:0] d,
    output [7:0] min
    );
    wire [7:0] min_ab, min_cd;
    assign min_ab = (a < b) ? a : b;
    assign min_cd = (c < d) ? c : d;
    assign min    = (min_ab < min_cd) ? min_ab : min_cd;
endmodule
```

Figure 8: design code

```
module tb_min_value;

    reg  [7:0] a, b, c, d;
    wire [7:0] min;

    min_value uut ( .a(a) , .b(b) , .c(c) , .d(d) , .min(min) );

    initial begin
        $display(" a    b    c    d    min");
        $monitor("%d    %d    %d    %d    %d", a, b, c, d, min);

        a = 8'd10; b = 8'd20; c = 8'd5;  d = 8'd15; #10;
        a = 8'd100; b = 8'd50; c = 8'd75; d = 8'd25; #10;
        a = 8'd1;  b = 8'd2;  c = 8'd3;  d = 8'd4;  #10;
        a = 8'd255; b = 8'd255; c = 8'd0; d = 8'd128; #10;

        $finish;
    end
endmodule
```

Figure 9: testbench code

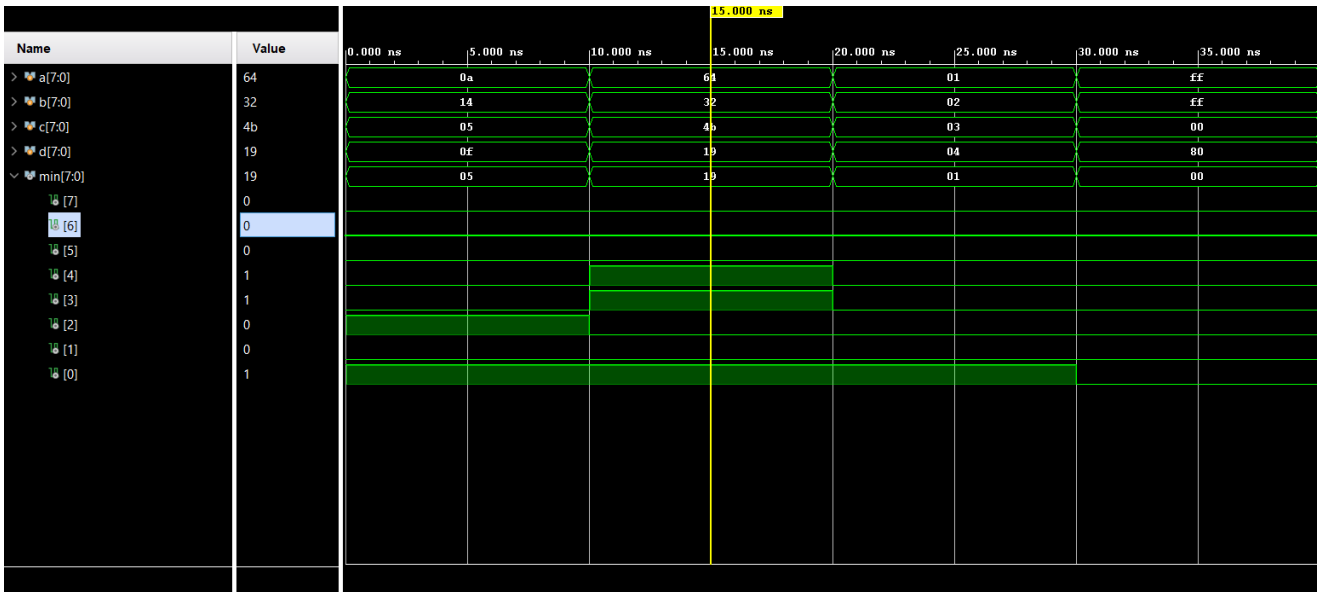

Figure 10: Waveform-3

Explanation

- The module determines the minimum value among four 8-bit inputs **a**, **b**, **c**, and **d**.
- The comparison is performed using conditional operators, where each value is compared step-by-step to find the smallest among them. The final minimum value is assigned to the output **min**.

- The testbench verifies the functionality by applying different combinations of input values. A delay is introduced between input changes to clearly observe the output in the waveform.
- The **$monitor** statement continuously displays the values of inputs and the resulting minimum value, and the simulation is terminated using **$finish**.

Observations

The waveform shows the minimum value among the four inputs at different time intervals.

- For **a = 0A**, **b = 14**, **c = 05**, **d = 0F**, the minimum value is **05**.
- For **a = 64**, **b = 32**, **c = 4B**, **d = 19**, the minimum value is **19**.
- For **a = 01**, **b = 02**, **c = 03**, **d = 04**, the minimum value is **01**.
- For **a = FF**, **b = FF**, **c = 00**, **d = 80**, the minimum value is **00**.

Thus, the waveform verifies that the module correctly identifies the minimum value among the given inputs.

4 Question 4

3-bit Population Count

```
module population_count(
    input [2:0] in,
    output [1:0] count
    );
    assign count = in[0] + in[1] + in[2];
endmodule
```

Figure 11: design code

```
module tb_population_count;

    reg  [2:0] in;
    wire [1:0] count;

    population_count uut ( .in(in) , .count(count) );

    initial begin
        $display(" in     count");
        $monitor("%b      %d", in, count);

        in = 3'b111; #10;
        in = 3'b000; #10;
        in = 3'b010; #10;
        in = 3'b101; #10;

        $finish;
    end
endmodule
```

Figure 12: testbench code

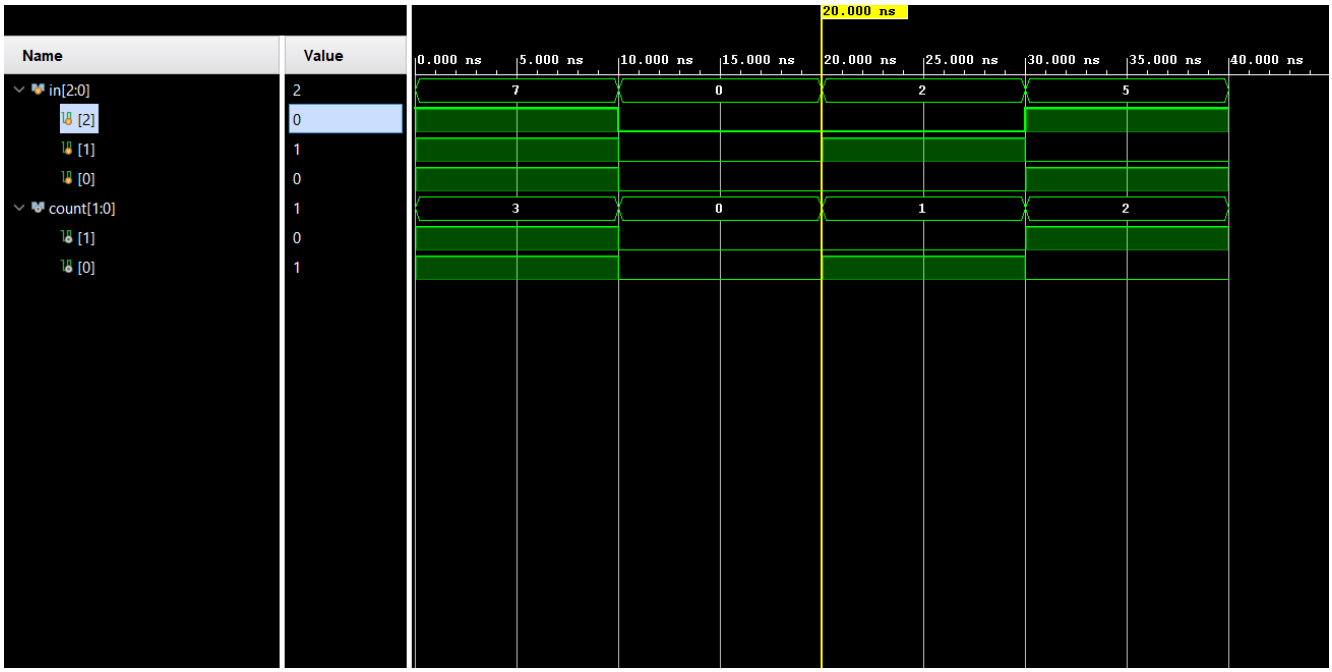

Figure 13: Waveform-4

Explanation

- The population count module counts the number of 1s present in a 3-bit input **in**. This is implemented using a continuous assignment statement, where all input bits are added together.
- The result is stored in the 2-bit output **count**, which represents the total number of 1s.
- The testbench verifies the functionality by applying different input combinations to **in**. A delay of **#10** is introduced between input changes to clearly observe the output.
- The **$monitor** statement continuously displays the input and corresponding count value, and the simulation is terminated using **$finish**.

Observations

The waveform shows the number of 1s present in the input **in**.

- For **in** = **111**, number of 1s = 3.
- For **in** = **000**, number of 1s = 0.
- For **in** = **010**, number of 1s = 1.
- For **in** = **101**, number of 1s = 2.

Thus, the waveform verifies that the population count module correctly counts the number of 1s in the input.

5 Question 5

8-bit Even Parity Generator

```
module even_parity_8bit(
    input [7:0] data,
    output parity
    );
   assign parity = data[7] ^ data[6] ^ data[5] ^ data[4] ^ data[3] ^ data[2] ^ data[1] ^ data[0];
endmodule
```

Figure 14: design code

```
module tb_even_parity_8bit;

reg  [7:0] data;
wire parity;

even_parity_8bit uut (.data(data),.parity(parity));

initial begin
    $display("   data        parity");
    $monitor("  %b       %b", data, parity);
    data = 8'b00000000; #5;
    data = 8'b00000111; #5;
    data = 8'b00000101; #5;
    data = 8'b10101010; #5;

    $finish;

end

endmodule
```

Figure 15: testbench code

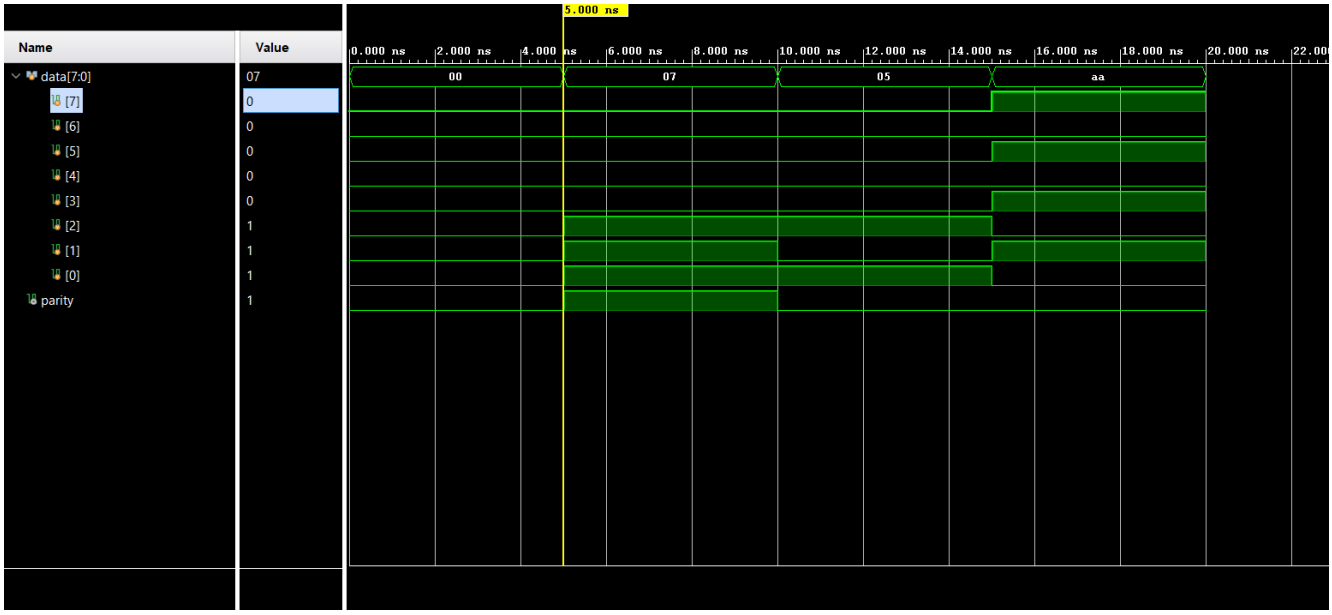

Figure 16: Waveform-5

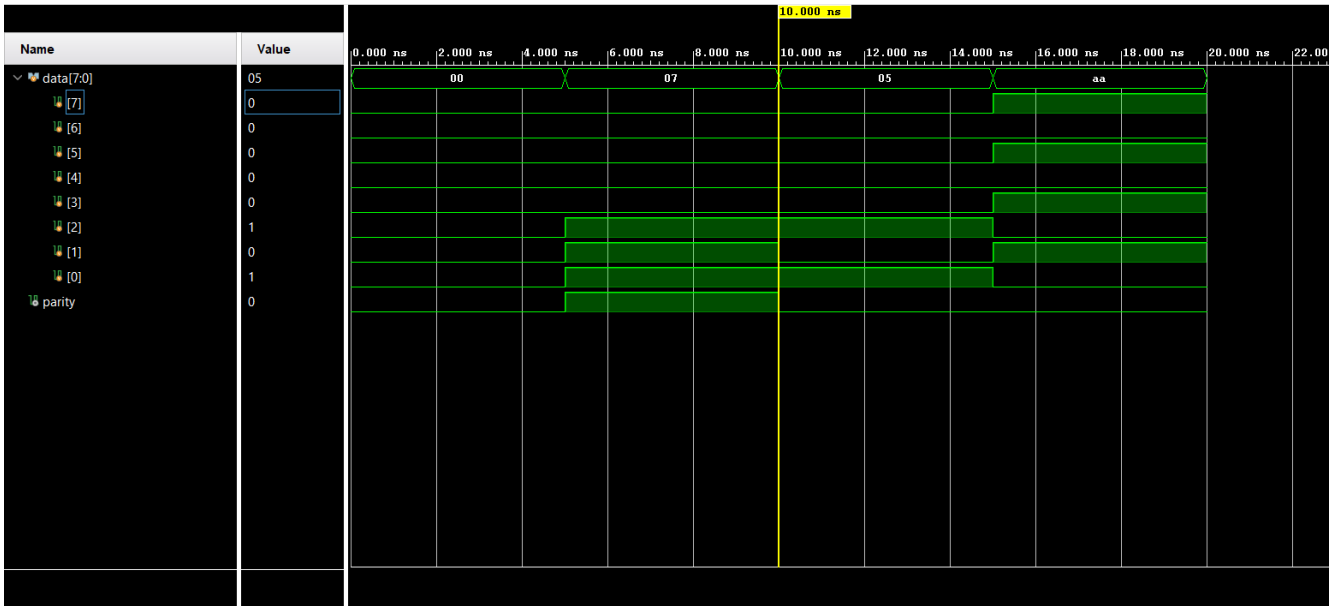

Figure 17: Waveform-5(1)

Explanation

- The 8-bit even parity generator is implemented using a continuous assignment statement, where all bits of the input **data[7:0]** are XORed together to produce the output **parity**.
- The XOR operation results in logic 1 when the number of 1s in the input is odd and 0 when it is even, thereby indicating the parity of the input.
- In the testbench code , A delay of **#5** is introduced between successive inputs to observe changes clearly in the waveform.
- The **$monitor** statement displays the input and output values during simulation, and the simulation ends using **$finish**.

Observations

The waveform shows the variation of input **data** and output **parity** with time.

At different time intervals:

- At 0–5 ns: **data = 00000000**, number of 1s = 0 (even), hence **parity = 0**.
- At 5–10 ns: **data = 00000111**, number of 1s = 3 (odd), hence **parity = 1**.
- At 10–15 ns: **data = 00000101**, number of 1s = 2 (even), hence **parity = 0**.
- At 15–20 ns: **data = 10101010**, number of 1s = 4 (even), hence **parity = 0**.

Thus, the waveform verifies that the $8-$ bit even parity generator produces the correct output.

6 Question 6

9-to-1 Multiplexer

```
module mux9to1(
    input  [15:0] a, b, c, d, e, f, g, h, i,
    input  [3:0]  sel,
    output [15:0] out
    );
    assign out = (sel == 4'd0) ? a :
                 (sel == 4'd1) ? b :
                 (sel == 4'd2) ? c :
                 (sel == 4'd3) ? d :
                 (sel == 4'd4) ? e :
                 (sel == 4'd5) ? f :
                 (sel == 4'd6) ? g :
                 (sel == 4'd7) ? h :
                 (sel == 4'd8) ? i :
                 16'hFFFF;
endmodule
```

Figure 18: design code

```
module tb_mux9to1;

reg [15:0] a, b, c, d, e, f, g, h, i;
reg [3:0]  sel;
wire [15:0] out;

mux_9to1 uut ( .a(a) , .b(b) , .c(c) , .d(d) , .e(e) , .f(f) , .g(g) , .h(h) , .i(i) , .sel(sel) , .out(out) );

initial begin
        $display(" sel   out");
        $monitor("%d     %h", sel, out);

a = 16'h0001 ; b = 16'h0002 ; c = 16'h0004 ; d = 16'h0008 ; e = 16'h0010;
f = 16'h0020 ; g = 16'h0040 ; h = 16'h0080 ; i = 16'h0100;

sel = 4'd0; #10;
sel = 4'd3; #10;
sel = 4'd6; #10;
sel = 4'd9; #10;

        $finish;
    end
endmodule
```

Figure 19: testbench code

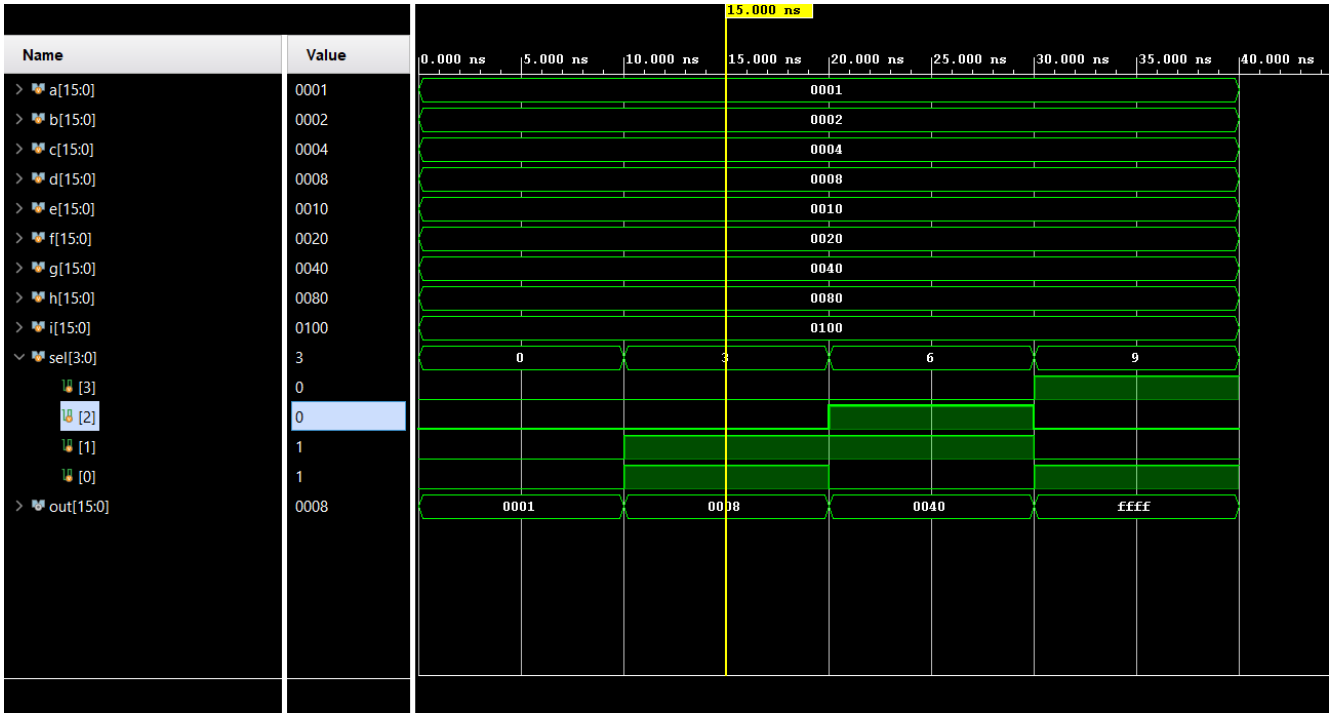

Figure 20: Waveform-6

Explanation

- The 9-to-1 multiplexer module selects one of nine 16-bit input signals based on the value of the 4-bit select signal **sel**. A nested conditional operator is used in the continuous assignment statement to compare the select value and assign the corresponding input to the output **out**.
- If **sel** takes values from 0 to 8, the output follows the respective input **a** to **i**. For any other value of **sel**, the output is assigned a default value of **16'hFFFF**.
- The testbench verifies the functionality by assigning distinct values to all inputs and applying different select values. A delay of **#10** is used between changes to observe the output clearly.
- The **$monitor** statement continuously displays the select value and corresponding output, and the simulation is terminated using **$finish**.

Observations

The waveform shows the output selection based on the value of **sel**.

- For **sel** = **0**, the output is **0001** (input **a**).
- For **sel** = **3**, the output is **0008** (input **d**).
- For **sel** = **6**, the output is **0040** (input **g**).
- For **sel** = **9**, the output is **FFFF** (default case).

Thus, the waveform verifies that the multiplexer correctly selects the input based on the select signal and handles invalid cases using a default output.